

JAVA COLLECTIONS FRAMEWORK

The Java Collections Framework (JCF) is a set of classes and interfaces that implement commonly reusable data structures. It is similar to the Standard Template Library (STL) in C++. Let's explore the various collections and utilities provided by Java in detail.

1. CUSTOM CLASSES

In Java, collections can store custom objects, allowing you to define your own classes and use them within collections such as List, Set, and Map. Let's take a simple class Person as an example:

```
class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + '}';
    }
}
```

Now, you can store instances of this Person class in a collection like an ArrayList:

```
List<Person> people = new ArrayList<>();
people.add(new Person("Alice", 30));
people.add(new Person("Bob", 25));
```

```
for (Person p : people) {  
    System.out.println(p);  
}
```

2. COLLECTIONS

a. List

A List is an ordered collection that allows duplicate elements. It provides positional access and is commonly used in scenarios where order matters.

i. ArrayList

An ArrayList is a resizable array implementation of the List interface. It offers fast random access but slower insertions and deletions as elements need to be shifted.

```
List<String> fruits = new ArrayList<>();  
fruits.add("Apple");  
fruits.add("Banana");  
fruits.add("Orange");
```

```
System.out.println("First fruit: " + fruits.get(0)); // Output: Apple
```

ii. LinkedList

LinkedList is a doubly linked list implementation of the List interface. It provides fast insertions and deletions but slower random access compared to ArrayList.

```
List<String> cars = new LinkedList<>();  
cars.add("Ford");  
cars.add("BMW");  
cars.add(0, "Tesla"); // Insert at the beginning
```

```
System.out.println("Cars: " + cars);
```

iii. Stack

Stack is a subclass of Vector that implements a last-in, first-out (LIFO) stack of elements. It provides typical stack operations like push() and pop().

```
Stack<String> history = new Stack<>();  
history.push("Page 1");
```

```
history.push("Page 2");
```

```
System.out.println("Popped: " + history.pop()); // Output: Page 2
```

iv. Vector

Vector is similar to ArrayList but is synchronized, meaning it is thread-safe for multi-threaded environments.

```
List<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);
```

```
System.out.println("Vector elements: " + numbers);
```

b. Set

A Set is a collection that does not allow duplicate elements. It is useful when you need to store unique elements.

i. HashSet

HashSet is an implementation of the Set interface that uses a hash table for storage. It provides constant time performance for basic operations like add and remove.

```
Set<String> uniqueWords = new HashSet<>();  
uniqueWords.add("hello");  
uniqueWords.add("world");  
uniqueWords.add("hello"); // Duplicate, will be ignored
```

```
System.out.println("Unique words: " + uniqueWords);
```

ii. TreeSet

TreeSet is an implementation of the Set interface that stores elements in a sorted order using a red-black tree. The elements are sorted based on their natural ordering or a custom comparator.

```
Set<Integer> sortedNumbers = new TreeSet<>();  
sortedNumbers.add(5);  
sortedNumbers.add(2);  
sortedNumbers.add(8);
```

```
System.out.println("Sorted numbers: " + sortedNumbers);
```

c. Queue

A Queue is a collection that follows the first-in, first-out (FIFO) principle. It is commonly used in scenarios where elements are processed in the order they are added.

i. ArrayQueue

Java doesn't have a direct ArrayQueue, but you can implement a queue using an ArrayList. Alternatively, you can use LinkedList as a queue.

ii. LinkedList (as Queue)

LinkedList can be used as a queue since it implements both the List and Queue interfaces.

```
Queue<String> tasks = new LinkedList<>();  
tasks.offer("Task 1"); // Add element  
tasks.offer("Task 2");
```

```
System.out.println("Processing: " + tasks.poll()); // Remove and return he
```

iii. PriorityQueue

PriorityQueue is a queue that orders elements according to their natural ordering or a custom comparator. Elements with higher priority are processed first.

```
Queue<Integer> pq = new PriorityQueue<>();  
pq.offer(4);  
pq.offer(1);  
pq.offer(7);
```

```
System.out.println("Priority queue head: " + pq.peek()); // Output: 1
```

3. MAP

A Map is a collection that maps keys to values. It does not allow duplicate keys, but multiple keys can map to the same value.

a. HashMap

HashMap is an implementation of the Map interface that uses a hash table for storage. It allows null keys and values.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);
ages.put("Bob", 25);
ages.put("Charlie", 35);
```

```
System.out.println("Bob's age: " + ages.get("Bob")); // Output: 25
```

b. TreeMap

TreeMap is a red-black tree-based implementation of the Map interface. It stores entries in sorted order based on keys.

```
Map<String, Integer> sortedAges = new TreeMap<>();
sortedAges.put("Charlie", 35);
sortedAges.put("Alice", 30);
sortedAges.put("Bob", 25);
```

```
System.out.println("Sorted by name: " + sortedAges);
```

4. ITERATOR

An Iterator allows you to traverse through a collection. ListIterator is a special type of iterator for List collections.

a. ListIterator

ListIterator allows bidirectional traversal of a list, i.e., both forward and backward.

```
List<String> colors = new ArrayList<>();
colors.add("Red");
colors.add("Green");
colors.add("Blue");
```

```
ListIterator<String> it = colors.listIterator();
while (it.hasNext()) {
    System.out.println("Forward: " + it.next());
}
```

```
while (it.hasPrevious()) {  
    System.out.println("Backward: " + it.previous());  
}
```

5. CUSTOM COMPARATOR

A Comparator allows you to define custom sorting logic for collections. You can use it to sort objects based on specific attributes.

```
// Assuming the Person class from section 1  
List<Person> peopleList = new ArrayList<>();  
peopleList.add(new Person("Alice", 30));  
peopleList.add(new Person("Bob", 25));  
peopleList.add(new Person("Charlie", 35));  
  
// Sort by age  
Collections.sort(peopleList, new Comparator<Person>>() {  
    @Override  
    public int compare(Person p1, Person p2) {  
        return Integer.compare(p1.getAge(), p2.getAge());  
    }  
});  
  
System.out.println("Sorted by age: " + peopleList);
```

6. COMMON ALGORITHMS

Java provides several utility methods through the Collections and Arrays classes:

`Collections.sort(list);` - Sorts a list in natural order.

```
List<Integer> numbersToSort = Arrays.asList(5, 1, 4, 2, 8);  
Collections.sort(numbersToSort);  
System.out.println("Sorted list: " + numbersToSort);
```

`Collections.max(list);` - Returns the maximum element from the list.

```
List<Integer> numbersForMax = Arrays.asList(5, 1, 4, 2, 8);  
System.out.println("Max element: " + Collections.max(numbersForMax));
```

`Collections.min(list);` - Returns the minimum element from the list.

```
List<Integer> numbersForMin = Arrays.asList(5, 1, 4, 2, 8);  
System.out.println("Min element: " + Collections.min(numbersForMin));
```

`Collections.reverse(list);` - Reverses the order of elements in the list.

```
List<String> colorsToReverse = new ArrayList<>(Arrays.asList("Red", "Green", "Blue", "Yellow", "Purple"));  
Collections.reverse(colorsToReverse);  
System.out.println("Reversed list: " + colorsToReverse);
```

`Arrays.sort(array);` - Sorts the elements of an array.

```
int[] numbersArray = {5, 1, 4, 2, 8};  
Arrays.sort(numbersArray);  
System.out.println("Sorted array: " + Arrays.toString(numbersArray));
```

`Collections.frequency(list, element);` - Returns the frequency of the element in the list.

```
List<String> fruitsList = Arrays.asList("Apple", "Banana", "Apple", "Orange", "Apple");  
System.out.println("Frequency of Apple: " + Collections.frequency(fruitsList, "Apple"));
```

`Collections.binarySearch(list, key);` - Performs a binary search for the key in the list (must be sorted).

```
List<Integer> sortedList = Arrays.asList(1, 2, 4, 5, 8);  
int index = Collections.binarySearch(sortedList, 4);  
System.out.println("Index of 4: " + index);
```

`Math.pow(base, exponent);` - Returns the result of raising the base to the power of the exponent.

```
double result = Math.pow(2, 3);  
System.out.println("2 to the power of 3 is: " + result);
```

CONCLUSION

The Java Collections Framework provides a robust set of classes and interfaces to handle data structures like lists, sets, queues, and maps. By understanding and utilizing these collections, developers can write efficient, maintainable, and scalable applications. In addition, utility methods provided by the `Collections` and `Arrays` classes simplify common operations such as sorting and searching, making Java a powerful tool for managing data.

Would you like to explore any of these Java Collections or algorithms in more detail?